
Sprawozdanie z laboratorium

Wydanie 0.0.1

Michał Bednarczyk

15 cze 2026

Spis treści:

1	Wprowadzenie	3
2	Badania literaturowe	5
2.1	Kopie zapasowe i odzyskiwanie danych	5
3	Dokumentacja bazy danych – Sklep elektroniczny	9
3.1	Wybór zagadnienia i identyfikacja danych	9
3.2	Opis procesów i towarzyszących im danych	9
3.3	Prototypowy plik JSON	10
3.4	Modelowanie koncektualne i logiczne	10

Add your content using reStructuredText syntax. See the [reStructuredText](#) documentation for details.

ROZDZIAŁ 1

Wprowadzenie

2.1 Kopie zapasowe i odzyskiwanie danych

2.1.1 Wstęp

Kopie zapasowe w PostgreSQL można podzielić na dwa główne nurty: kopie logiczne wykonywane narzędziami `pg_dump` i `pg_dumpall` [cite: 3] oraz kopie fizyczne całego klastra wykonywane m.in. przez `pg_basebackup` wraz z archiwizacją WAL dla odzyskiwania punktowego PITR (Point-in-Time Recovery)[cite: 3, 4]. Kopie logiczne są wygodne do odtwarzania pojedynczych obiektów i pojedynczych baz, natomiast kopie fizyczne są podstawą pełnego odtworzenia instancji, tablespaces i odzyskiwania po awariach[cite: 5].

2.1.2 Tworzenie kopii zapasowej całego systemu PostgreSQL - mechanizmy wbudowane

Najprostszym wbudowanym sposobem wykonania kopii całego klastra jest `pg_dumpall`, które eksportuje wszystkie bazy w klastrze oraz obiekty globalne wspólne dla całej instancji, takie jak role i przestrzenie tabel[cite: 7]. Taki zrzut ma postać tekstowego skryptu SQL i odtwarza się go zwykle przez `psql`, ale metoda ta jest logiczna, więc nie daje możliwości odzyskiwania punktowego przez WAL[cite: 8].

Drugim, ważniejszym z punktu widzenia odporności na awarie mechanizmem jest `pg_basebackup`, które tworzy fizyczną kopię działającego klastra bez blokowania normalnej pracy klientów[cite: 10]. Narzędzie to wykonuje kopię całego klastra, a nie pojedynczych baz czy tabel, może pracować w formacie katalogowym lub `tar` i może dołączać wymagane pliki WAL metodą `stream`, `fetch` albo `none`[cite: 11]. Dokumentacja podkreśla też, że taka kopia może służyć zarówno do PITR, jak i do zbudowania serwera standby[cite: 12].

Aby pełna kopia fizyczna była użyteczna w scenariuszu odtwarzania do wybranego momentu, należy włączyć archiwizację WAL przez ustawienie co najmniej `wal_level = replica`, `archive_mode = on` oraz poprawnego `archive_command` albo `archive_library``[cite: 13]. PostgreSQL zapisuje każdą zmianę w logu WAL, a po odtworzeniu kopii bazowej można odtworzyć kolejne segmenty WAL, aby dojść do stanu bieżącego lub do wskazanego punktu w czasie[cite: 14].

Przykładowe polecenia:

```
# logiczna kopia całego klastra
pg_dumpall -U postgres --clean --file=cluster.sql

# fizyczna kopia całego klastra z dołączeniem WAL
pg_basebackup -h dbserver -D /backup/base -Fp -X stream -P
```

W praktyce do dokumentacji laboratoryjnej warto zaznaczyć, że `pg_dumpall` lepiej nadaje się do migracji i prostych kopii administracyjnych, a `pg_basebackup` z archiwizacją WAL do scenariuszy disaster recovery[cite: 21].

2.1.3 Tworzenie kopii zapasowych poszczególnych baz danych - mechanizmy wbudowane

Do wykonania kopii pojedynczej bazy służy `pg_dump`, które tworzy spójny zrzut nawet wtedy, gdy baza jest równocześnie używana przez innych użytkowników, i nie blokuje zwykłych operacji odczytu ani zapisu[cite: 24]. Narzędzie obsługuje format tekstowy, custom, directory oraz tar, przy czym formaty archiwalne współpracują z `pg_restore` i pozwalają na selektywne odtwarzanie obiektów[cite: 24].

Istotne jest to, że `pg_dump` wykonuje kopię tylko jednej bazy danych [cite: 26]; do eksportu całego klastra lub obiektów globalnych należy użyć `pg_dumpall` [cite: 27]. Format `custom` (`-Fc`) i `directory` (`-Fd`) są najbardziej elastyczne, bo pozwalają wybierać odtwarzane elementy, zmieniać kolejność odtwarzania, a w części scenariuszy także korzystać z odtwarzania równoległego[cite: 27].

Przykładowe polecenia:

```
# kopia pojedynczej bazy w formacie custom
pg_dump -U postgres -d labdb -Fc -f /backup/labdb.dump

# kopia pojedynczej bazy jako skrypt SQL
pg_dump -U postgres -d labdb -Fp -f /backup/labdb.sql

# odtworzenie archiwum custom
pg_restore -d labdb_restored /backup/labdb.dump
```

Jeżeli celem jest ochrona wybranych schematów lub tabel, `pg_dump` pozwala zawęzić zakres eksportu przez opcje takie jak `-n` dla schematu i `-t` dla tabeli[cite: 36]. Trzeba jednak zaznaczyć, że wybiórczy zrzut nie gwarantuje kompletności zależności wszystkich obiektów w nowym, pustym środowisku, jeśli pominięto elementy, od których zależy odtwarzany obiekt[cite: 37].

2.1.4 Odzyskiwanie usuniętych lub uszkodzonych danych

Sposób odzyskiwania zależy od tego, czy problem dotyczy pojedynczego obiektu logicznego, całej bazy, przestrzeni tabel, czy fizycznego uszkodzenia danych[cite: 40]. PostgreSQL rozróżnia w praktyce odzyskiwanie logiczne z plików dump oraz odzyskiwanie fizyczne z kopii bazowej i archiwum WAL[cite: 41].

Odtwarzanie tabel i innych obiektów logicznych

Jeżeli wcześniej wykonano kopię `pg_dump` w formacie archiwalnym, można odtworzyć pojedynczą tabelę lub inny obiekt selektywnie przy pomocy `pg_restore`, bez przywracania całej bazy[cite: 43]. To jest najprostszy wariant odzyskania przypadkowo usuniętej tabeli, o ile odpowiedni obiekt znajdował się w logicznej kopii zapasowej[cite: 43].

Przykład:

```
pg_restore -d labdb -t public.wyniki /backup/labdb.dump
```

Odtwarzanie usuniętej bazy danych

Usuniętą bazę można najłatwiej odtworzyć z wcześniejszego zrzutu `pg_dump` tej konkretnej bazy albo z `pg_dumpall`, jeżeli wykonywano kopię całego klastra [cite: 48]. W przypadku wymogu odtworzenia dokładnie do chwili sprzed usunięcia, potrzebna jest kopia fizyczna oraz archiwum WAL, ponieważ same zrzuty logiczne nie są częścią rozwiązania continuous archiving i nie wspierają PITR [cite: 49].

Odtwarzanie przestrzeni tabel i pełnego klastra

Przestrzenie tabel są obiektami globalnymi klastra, dlatego ich definicje są uwzględniane przez `pg_dumpall`, a przy kopiach fizycznych `pg_basebackup` zachowuje także ich strukturę i mapowanie [cite: 52]. Dokumentacja `pg_basebackup` wskazuje, że przy pracy z tablespaces można użyć mapowania `--tablespace-mapping`, a przy odtwarzaniu trzeba zweryfikować poprawność dowiązań symbolicznych i lokalizacji danych [cite: 52].

Przy fizycznym uszkodzeniu danych, katalogu PGDATA albo tablespaces standardowa procedura obejmuje odtworzenie kopii bazowej, usunięcie starych plików danych, odtworzenie katalogów danych i tablespaces, skonfigurowanie `restore_command`, utworzenie pliku `recovery.signal`, a następnie ponowne uruchomienie serwera w trybie recovery [cite: 54]. PostgreSQL podczas odtwarzania odczytuje wymagane segmenty WAL i może dojść do stanu bieżącego albo zatrzymać się na wybranym punkcie odzyskiwania [cite: 55].

PITR i odzyskiwanie stanu sprzed błędu

Najważniejszym mechanizmem odzyskiwania po usunięciu danych operacyjnych jest PITR, czyli przywrócenie systemu do chwili sprzed błędnej operacji, na przykład `DROP TABLE` lub `DROP DATABASE` [cite: 58]. W takim scenariuszu odtwarza się kopię bazową, konfiguruje `restore_command`, a następnie ustawia cel odzyskiwania, np. przez czas, nazwany punkt przywracania lub identyfikator transakcji [cite: 59, 60].

To rozwiązanie ma jednak ważne ograniczenie: nie służy do odzyskania pojedynczej tabeli „w miejscu” bez wpływu na resztę systemu, lecz do odtworzenia całego klastra do wcześniejszego stanu [cite: 61]. Dlatego w praktyce laboratoryjnej często łączy się oba podejścia: regularne `pg_dump` dla obiektów logicznych i `pg_basebackup` z archiwizacją WAL dla pełnego recovery [cite: 61].

2.1.5 Dedykowane oprogramowanie i skrypty zewnętrzne do automatyzacji

Wbudowane narzędzia PostgreSQL są wystarczające do ręcznego wykonywania kopii, ale w środowisku produkcyjnym często wykorzystuje się oprogramowanie automatyzujące harmonogramy, retencję, walidację i odtwarzanie [cite: 63]. Dwa najczęściej wskazywane rozwiązania to Barman i pgBackRest, przy czym dostępna dokumentacja Barman bezpośrednio opisuje obsługę pełnych i przyrostowych kopii, archiwizacji WAL oraz automatycznego uruchamiania backupów [cite: 64].

Barman wykonuje kopie całego serwera PostgreSQL i wymaga poprawnej archiwizacji WAL, oferując różne metody kopii, w tym streaming przez `pg_basebackup`, `rsync` oraz backupy snapshotowe w chmurze [cite: 66, 67]. Dokumentacja podaje też wsparcie dla backupów przyrostowych, limitowania pasma, kompresji i pracy z harmonogramem przez zewnętrzne mechanizmy systemowe, np. cron [cite: 67, 68].

Przykładowe zastosowania narzędzi zewnętrznych:

- **Barman:** centralne repozytorium backupów, archiwizacja WAL, backupy pełne i przyrostowe, odzyskiwanie całych instancji [cite: 70].
- **pgBackRest:** popularne narzędzie do wydajnych backupów i retencji, często używane w środowiskach HA oraz większych instalacjach PostgreSQL [cite: 71].
- **Własne skrypty Bash/PowerShell:** wywołanie `pg_dump`, `pg_dumpall` lub `pg_basebackup`, kompresja plików, rotacja katalogów, logowanie i wysyłka wyników przez e-mail lub do systemu monitoringu [cite: 72].

Przykładowy prosty skrypt automatyzujący kopię jednej bazy:

```
#!/bin/bash
DATA=$(date +%F_%H-%M)
KATALOG=/backup/postgres
BAZA=labdb

mkdir -p "$KATALOG"
pg_dump -U postgres -d "$BAZA" -Fc -f "$KATALOG/${BAZA}_${DATA}.dump"
find "$KATALOG" -type f -name "${BAZA}*.dump" -mtime +7 -delete
```

Taki skrypt jest prosty, ale nie zapewnia pełnego disaster recovery, bo nie obejmuje archiwizacji WAL i nie chroni całego klastra[cite: 81]. Z tego powodu dla systemów krytycznych bardziej właściwe są narzędzia klasy Barman lub pgBackRest, które porządkują pełny proces backupu, retencji, testów i odtwarzania[cite: 81, 82].

2.1.6 Wnioski

W rozdziale warto wyraźnie rozróżnić kopie logiczne i fizyczne, bo odpowiadają one na inne potrzeby eksploatacyjne[cite: 85]. `pg_dump` i `pg_dumpall` są najlepsze do prostych eksportów i selektywnego odtwarzania, natomiast `pg_basebackup` z archiwizacją WAL jest podstawą odzyskiwania po poważnej awarii i realizacji PITR[cite: 86].

Dobłą praktyką jest także zapisanie, że skuteczna strategia backupu nie kończy się na wykonaniu kopii, ale obejmuje testy odtwarzania, kontrolę retencji, ochronę plików backupu oraz automatyzację procesu[cite: 88]. W laboratorium można więc pokazać zarówno proste polecenia wbudowane, jak i przykład narzędzia zewnętrznego, które organizuje cały cykl tworzenia i odtwarzania kopii zapasowych[cite: 88, 89].

Dokumentacja bazy danych – Sklep elektroniczny

Autorzy: Norbert Antonovitch, Michał Bednarczyk, Jan Balazs de Borbatviz

3.1 Wybór zagadnienia i identyfikacja danych

Tematem projektu jest podstawowa baza danych obsługująca sklep internetowy ze sprzętem elektronicznym. Baza danych zawiera następujące grupy danych:

- **Klienci:** unikalny identyfikator, imię, nazwisko, adres e-mail, numer telefonu oraz podstawowe dane adresowe do wysyłki: ulica, miasto, kod pocztowy.
- **Produkty:** unikalny identyfikator, nazwa sprzętu, cena jednostkowa oraz przypisana kategoria.
- **Kategorie:** identyfikator kategorii oraz jej nazwa, na przykład Laptopy lub Smartfony.
- **Zamówienia:** identyfikator zamówienia, data złożenia, status, na przykład nowe lub zrealizowane, oraz powiązanie z konkretnym klientem.
- **Szczegóły zamówienia:** pozycje przypisane do konkretnego zamówienia, określające jaki produkt został kupiony, w jakiej ilości oraz po jakiej cenie.

3.2 Opis procesów i towarzyszących im danych

1. **Zarządzanie produktami:** sklep oferuje asortyment przypisany do konkretnych kategorii, na przykład laptopy i smartfony. Każdy produkt ma przypisaną nazwę, cenę oraz identyfikator producenta.
2. **Zarządzanie użytkownikami:** klienci rejestrują się w systemie, podając podstawowe dane, takie jak imię, nazwisko i e-mail, oraz pojedynczy główny adres zamieszkania wykorzystywany do wysyłki.
3. **Realizacja zamówień:** zalogowany klient składa zamówienie na wybrane produkty. System tworzy nagłówek zamówienia zawierający datę i status oraz przypisuje do niego poszczególne produkty wraz z ich ilością.

3.3 Prototypowy plik JSON

Poniżej przedstawiono prototypowy plik w formacie JSON, zawierający jeden pełny dokument reprezentujący złożone zamówienie. Pozwala to zweryfikować kompletność gromadzonych i przetwarzanych informacji przed przejściem do modelowania konceptualnego.

```
{
  "id_zamowienia": 1,
  "data_zlozenia": "2026-05-14T10:15:30Z",
  "status": "Nowe",
  "klient": {
    "id_klienta": 4096,
    "imie": "Jan",
    "nazwisko": "Kowalski",
    "email": "student01@example.com",
    "telefon": "123456789",
    "ulica": "Armii Krajowej 78",
    "kod_pocztowy": "58-300",
    "miasto": "Walbrzych"
  },
  "pozycje": [
    {
      "id_produktu": 10543,
      "nazwa": "Laptop ASUS",
      "kategoria": "Laptopy",
      "ilosc": 1,
      "cena_historyczna": 6299.99
    }
  ]
}
```

3.3.1 Dobór typów danych

Tabela 1: Mapowanie typów danych dla SQLite i PostgreSQL

Atrybut danych	SQLite	PostgreSQL
Identyfikatory (id_...)	INTEGER	SERIAL lub INTEGER
Napisy krótkie (imie, nazwisko, miasto)	TEXT	VARCHAR(50)
Napisy długie (ulica, email, nazwa)	TEXT	VARCHAR(255)
Stałe ciągi znaków (telefon, kod_pocztowy)	TEXT	VARCHAR(15)
Cena (cena_bazowa, cena_historyczna)	FLOAT	NUMERIC(10,2)
Ilość (ilosc)	INTEGER	SMALLINT
Data i czas (data_zlozenia)	TEXT	TIMESTAMP

3.4 Modelowanie konceptualne i logiczne

3.4.1 Model koncepcyjny

1. Encje mocne

- **Klient:** klucz główny (id_klienta), imię, nazwisko, e-mail, telefon, ulica, miasto, kod pocztowy.
- **Produkt:** klucz główny (id_produktu), nazwa, cena_bazowa.

- **Kategoria:** klucz główny (`id_kategorii`), nazwa.
- **Zamówienie:** klucz główny (`id_zamowienia`), `data_zlozenia`, `status`.

2. Encje słabe

- **Szczegóły zamówienia:** klucz obcy (`id_zamowienia`), klucz obcy (`id_produktu`), `ilość`, `cena_historyczna`. Jej identyfikacja opiera się wyłącznie na kluczach obcych pochodzących z encji silnych.

3. Opis związków

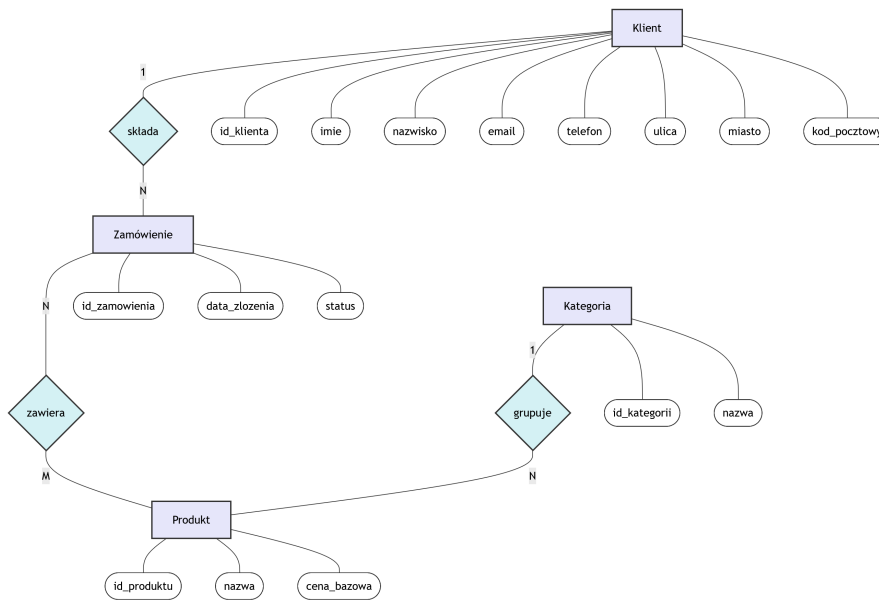
Zdefiniowano następujące relacje biznesowe między encjami:

- **Klient – Zamówienie (1:N):** klient składa zamówienie. Jeden klient może posiadać wiele zamówień, ale jedno konkretne zamówienie jest przypisane dokładnie do jednego klienta.
- **Kategoria – Produkt (1:N):** kategoria grupuje produkty. Kategoria może zawierać wiele produktów, ale dany produkt należy do jednej kategorii.
- **Zamówienie – Produkt (M:N):** zamówienie zawiera produkty. Pojedyncze zamówienie obejmuje wiele produktów, a dany produkt może pojawić się w wielu zamówieniach.

4. Związki niepoprawne

Wyeliminowano bezpośrednią relację między Klientem a Produktem, aby uniknąć pułapki połączeń. Bezpośrednie powiązanie gubi kontekst transakcji, na przykład datę i ilość zakupionego towaru, dlatego poprawna relacja musi zawsze przechodzić przez encję Zamówienie.

3.4.2 Model koncepcyjny – notacja Chena



3.4.3 Model logiczny i normalizacja

Główne działania podjęte podczas normalizacji:

1. **I Postać Normalna (1NF):** dane adresowe klienta, takie jak ulica, miasto i kod pocztowy, zostały wyodrębnione jako pojedyncze pola.
2. **II Postać Normalna (2NF):** zlikwidowano relację wiele-do-wielu (N:M) pomiędzy Zamówieniem a Produktem, wstawiając encję asocjacyjną **Szczegóły_zamówienia**. Jej klucz główny składa się z identyfikatora zamówienia oraz identyfikatora produktu.

3. **III Postać Normalna (3NF)**: usunięto zależności przechodnie. Nazwa kategorii została wydzielona do osobnej tabeli Kategorie. W tabeli Produkty przechowywany jest jedynie klucz obcy do kategorii. Ponadto zapewniono zapisywanie historycznej ceny w Szczegółach_zamówienia, aby modyfikacja cennika nie fałszowała archiwalnych rachunków.

3.4.4 Schemat notacji IE po normalizacji

